

Sistema Local de Perguntas e Respostas com Retrieval-Augmented Generation: Llama 3.2, ChromaDB e Web Scraping

Enrico Nunes

Universidade da Beira Interior, Covilhã, Portugal

Abstract

Este relatório descreve a conceção e a implementação de um sistema local de perguntas e respostas baseado em *Retrieval-Augmented Generation* (RAG). A partir de um único endereço Web definido pelo utilizador, o sistema faz *web scraping* do conteúdo, segmenta-o, gera *embeddings* com um modelo *Sentence-Transformers* e indexa-os numa base vetorial **ChromaDB** persistida em disco. As respostas são geradas pelo modelo **Llama 3.2**, servido localmente pelo **Ollama**, usando exclusivamente o contexto recuperado—uma estratégia explícita de mitigação de alucinações. Todo o processamento, com exceção da recolha inicial de dados, ocorre na própria máquina, sem chamadas a APIs externas, cumprindo o requisito de operação *offline*. A solução foi desenvolvida num *notebook* Jupyter organizado em onze etapas sequenciais e fundamenta-se diretamente nas aulas teóricas de Computação em Linguagem Natural da Universidade da Beira Interior.

Keywords

Retrieval-Augmented Generation, Llama 3.2, ChromaDB, Ollama, Sentence Transformers, Web Scraping

1. Introdução

O objetivo deste trabalho foi construir um sistema de perguntas e respostas completamente **local** sobre uma fonte de conhecimento arbitrária da Web. Em vez de enviar dados para serviços externos, toda a indexação, pesquisa semântica e geração de resposta decorre na própria máquina. A única etapa que necessita de ligação à Internet é a recolha inicial de conteúdo (*web scraping*); a partir desse momento, o sistema responde **sem qualquer dependência de rede**.

Um sistema RAG combina duas fases: (1) a *recuperação* dos excertos mais relevantes para uma pergunta e (2) a *geração* de uma resposta fundamentada nesse contexto [4]. Esta abordagem permite responder com base num corpus específico—neste caso, uma página Web indexada—em vez de depender apenas do conhecimento paramétrico do modelo de linguagem, reduzindo o problema das *alucinações* [5].

O sistema foi implementado num *notebook* Jupyter, deliberadamente dividido em etapas curtas e com lógica temporal, de modo a tornar transparente cada passo do *pipeline*. O código-fonte está disponível publicamente em¹. Este relatório começa por contextualizar as ferramentas e conceitos centrais (Secção 2), enquadra o trabalho nas aulas teóricas da unidade curricular (Secção 3), descreve a arquitetura (Secção 4) e detalha as onze etapas do *notebook* (Secção 5), terminando com uma demonstração real do sistema, a declaração de utilização de IA generativa e as conclusões.

2. Fundamentos e Ferramentas Centrais

Antes de descrever a implementação, importa contextualizar os temas que sustentam o trabalho. Estes são, simultaneamente, os conceitos teóricos e as ferramentas concretas que compõem o sistema.

Transformadores e modelos de linguagem. A arquitetura *Transformer*, introduzida por Vaswani et al. [1], substituiu a recorrência pelo mecanismo de *self-attention* e tornou-se a base de praticamente

UBI 2025/26, Mestrado em Engenharia Informática, Computação em Linguagem Natural

✉ enrico.nunes@ubi.pt (E. Nunes)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

¹<https://github.com/enriconunes/sistema-RAG-Local-Llama-ChromaDB>

todos os modelos de linguagem modernos. Sobre ela construiu-se o **BERT** [2], um modelo bidirecional pré-treinado por *masked language modeling*, que popularizou as representações contextuais de texto. Estes dois trabalhos são a fundação tanto dos modelos geradores quanto dos modelos de *embeddings* usados neste sistema.

Modelos de linguagem de grande escala e o Llama 3.2. Os *Large Language Models* (LLM) são modelos *Transformer* de grande dimensão capazes de gerar texto fluente. Neste trabalho utiliza-se o **Llama 3.2**, pertencente à família de modelos abertos da Meta descrita em *The Llama 3 Herd of Models* [3]. A escolha de um modelo aberto é decisiva: permite executá-lo **localmente**, sem custos por chamada e sem enviar dados para partes externas ao sistema. Essa execução local é assegurada pelo **Ollama** [8], um servidor que descarrega, gere e expõe modelos de linguagem na própria máquina através de um *endpoint* HTTP em localhost.

Embeddings e recuperação semântica. A recuperação em RAG assenta em *embeddings*: vetores numéricos que posicionam textos com significado semelhante próximos uns dos outros. Em vez da pesquisa por palavras-chave, usa-se *dense retrieval*, onde a pergunta e os documentos são projetados no mesmo espaço vetorial e comparados por similaridade.

Bases de dados vetoriais. Os *embeddings* têm de ser armazenados e pesquisados de forma eficiente. Para isso usa-se o **ChromaDB** [9], uma base de dados vetorial de código aberto, orientada a aplicações de IA, que persiste os vetores em disco (via SQLite) e permite recarregá-los posteriormente sem acesso à rede—uma propriedade essencial para o modo *offline*.

Retrieval-Augmented Generation e alucinações. O RAG [4] articula recuperação e geração para fundamentar as respostas em evidência textual concreta. O seu principal benefício é reduzir as *alucinações*—afirmações fluentes mas factualmente incorretas ou não suportadas—um problema sistematicamente documentado na literatura [5]. Ao obrigar o modelo a responder apenas com base no contexto recuperado, e a admitir quando a informação não está disponível, o sistema torna-se mais fiável e auditável.

Orquestração e recolha de dados. A composição do *pipeline* (recuperação, construção de *prompt* e geração) é feita com a *framework* **LangChain** [10], através da sua linguagem de composição declarativa (LCEL). A recolha inicial do conteúdo Web é feita com *requests* e **Beautiful Soup** [11], que faz *parsing* do HTML e extrai o texto relevante.

3. Enquadramento nas Aulas de Computação em Linguagem Natural

Todo o trabalho foi baseado nas aulas teóricas da unidade curricular de *Computação em Linguagem Natural* (UBI, MEI, 2025/26), lecionadas pelo Prof. João Cordeiro. Em concreto, os temas deste projeto decorrem diretamente de três aulas:

- **Aula Teórica 09 — *Masked Language Models: BERT*.** Apresenta os modelos bidirecionais baseados em transformadores, as representações contextuais e a motivação para a recuperação de informação e para o RAG, incluindo a discussão das alucinações dos modelos de linguagem. Fundamenta os conceitos de *embeddings* e de recuperação semântica usados no sistema.
- **Aula Teórica 10 — *Prompt Engineering + RAG*.** Introduz a engenharia de *prompts*, o *pipeline* de RAG (*loading*, *chunking*, *embedding* e *vector store*), a recolha de conteúdo da Web e a *framework* LangChain. Corresponde à espinha dorsal da fase de ingestão deste trabalho.
- **Aula Teórica 11 — *RAG, LangChain, Indexing, Topic Modeling*.** Demonstra a utilização do LangChain com diferentes modelos e bases de dados, nomeadamente **Llama 3.2** e **ChromaDB**, a construção de cadeias LCEL, os *loaders* e *text splitters*, e ainda a modelação de tópicos com LDA. É a aula mais próxima da solução aqui apresentada.

4. Arquitetura

A solução divide-se em dois fluxos bem delimitados, separados por uma fronteira *online* / *offline*:

1. **Ingestão** (executada uma vez, requer Internet): a partir de um URL, faz-se *web scraping* do conteúdo, segmenta-se em *chunks*, geram-se *embeddings* e indexam-se na ChromaDB, persistida em disco.
2. **Operação** (*offline*, sem APIs): dada uma pergunta, o *retriever* recupera os *chunks* mais relevantes da base, que são injetados num *prompt* e enviados ao Llama 3.2 (via Ollama) para produzir uma resposta ancorada e com indicação das fontes.

Esta separação é central: depois de a base vetorial estar em disco, a fase de operação não volta a aceder à rede, o que é demonstrado explicitamente na última etapa do *notebook* (modo *offline*).

5. As Onze Etapas do Sistema

O *notebook* está organizado de forma sequencial. A **Etapa 0** é a configuração centralizada; as **Etapas 1 a 11** constituem o *pipeline* propriamente dito, da verificação do ambiente até à operação *offline*. Descreve-se de seguida cada etapa, com a respetiva justificação técnica.

Etapa 0 — Configuração

Todos os parâmetros do sistema ficam reunidos num único bloco, para que mudar de fonte de conhecimento se resuma a alterar o URL e reexecutar o *notebook*.

```
URL = "https://pt.wikipedia.org/wiki/Processamento_de_linguagem_natural"
OLLAMA_MODEL = "llama3.2"
OLLAMA_BASE_URL = "http://localhost:11434"
EMBEDDING_MODEL = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
CHROMA_DIR = "chroma_db" # persistencia em disco
CHUNK_SIZE = 1000
CHUNK_OVERLAP = 100
TOP_K = 4 # chunks recuperados por pergunta
```

Etapa 1 — Verificação do ambiente

Antes de avançar, a função `verificar_ollama()` interroga o servidor local (*endpoint* /`api/tags`) para confirmar que o Ollama está acessível e que o modelo `llama3.2` está descarregado. Trata-se de um processo local, não de uma API remota. Esta verificação prévia evita falhas obscuras mais à frente e dá ao utilizador instruções claras de correção.

Etapa 2 — Web Scraping

É a **única** etapa que necessita de Internet. A função `extrair_conteudo(url)` faz o pedido HTTP com `requests` e analisa o HTML com BeautifulSoup [11], removendo elementos sem valor informativo (`script`, `style`, `nav`, `footer`, etc.), privilegiando o corpo principal do artigo e normalizando espaços e linhas.

```
soup = BeautifulSoup(resp.text, "html.parser")
for tag in soup(["script", "style", "nav", "footer", "header", "aside", "noscript", "form"]):
    tag.decompose()
principal = soup.find("main") or soup.find("article") or soup.body or soup
texto = principal.get_text(separator="\n")
```

Etapa 3 — Chunking (segmentação)

O texto extraído é demasiado grande para ser usado de uma só vez, pelo que é dividido em segmentos menores com o `RecursiveCharacterTextSplitter` (`chunk_size=1000`, `chunk_overlap=100`). A sobreposição (*overlap*) preserva o contexto na fronteira entre *chunks*, evitando que informação relevante fique artificialmente cortada. A cada *chunk* é atribuído um identificador, para mais tarde citar a proveniência da resposta.

Etapa 4 — Embeddings locais

Cada *chunk* é convertido num vetor que captura o seu significado, com um modelo *Sentence-Transformers* executado **localmente** (`HuggingFaceEmbeddings`). Os vetores são normalizados, o que torna a similaridade do cosseno equivalente ao produto interno.

Etapa 5 — Indexação no ChromaDB

Os *chunks* e respetivos vetores são guardados na ChromaDB com `Chroma.from_documents(...)`, que cria a coleção e persiste tudo em disco via SQLite [9]. Depois desta etapa, a base pode ser recarregada *offline* (Etapa 11), sem reprocessar a página.

```
vector_store = Chroma.from_documents(
    documents=chunks, embedding=embeddings,
    collection_name=COLLECTION_NAME, persist_directory=CHROMA_DIR,
)
```

Etapa 6 — Retriever

A partir da base cria-se um *retriever* com as `retriever(search_kwargs={"k": TOP_K})`, que devolve os *k* *chunks* mais semelhantes a cada pergunta, por similaridade no espaço de *embeddings*. O *notebook* mostra explicitamente *que* excertos são recuperados para uma consulta de exemplo—uma transparência que ajuda a compreender (e a defender) cada resposta.

Etapa 7 — Modelo de linguagem local (Ollama + Llama 3.2)

Carrega-se o Llama 3.2 [3] através do ChatOllama. Define-se `temperature=0` para tornar as respostas determinísticas e menos propensas a invenção.

Etapa 8 — Cadeia RAG (LCEL)

Compõe-se o *pipeline* em LCEL [10], ligando *retriever*, *prompt*, modelo e *parser* de saída:

```
rag_chain = (
    {"context": retriever | formatar_docs, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
```

O *prompt* de sistema é o coração da estratégia anti-alucinação: instrui o modelo a responder **exclusivamente** com base no contexto e, se a informação não constar, a devolver exatamente “*Não encontrei essa informação no conteúdo indexado.*” [5].

Etapa 9 — Perguntas e Respostas

A função `perguntar()` executa a cadeia e apresenta a resposta **acompanhada das fontes** (os *chunks* efetivamente utilizados), seguindo a ideia de citação de proveniência. Mostrar as fontes é o que distingue uma resposta verificável de uma afirmação opaca.

Etapa 10 — Demonstração anti-alucinação

Esta etapa contrasta, para a mesma pergunta, duas respostas: a do **modelo sozinho** (que responde a partir da memória paramétrica e pode inventar) e a do **sistema com RAG** (que se ancora no conteúdo indexado e, na ausência de evidência, recusa responder). É a materialização prática do problema das alucinações.

Etapa 11 — Modo Offline

Para provar o requisito de operação sem Internet, a função `carregar_sistema_offline()` **recarrega a ChromaDB já persistida em disco** (sem *scraping* nem APIs) e reconstrói a cadeia de resposta. É possível desligar a rede e executar apenas esta célula, obtendo na mesma respostas fundamentadas.

5.1. Demonstração: comportamento indexado vs. não indexado

A Figura 1 mostra o sistema em funcionamento real, com a fonte indexada sobre Processamento de Linguagem Natural (PLN). Ilustra os dois comportamentos que resumem todo o trabalho:

- **Pergunta não indexada** (“*qual é o atual presidente de portugal?*”): como a resposta não consta do conteúdo recuperado, o sistema **recusa-se a inventar** e devolve “*Não encontrei essa informação no conteúdo indexado.*”
- **Pergunta indexada** (“*o que é PNL*”): o sistema produz uma resposta correta e fundamentada no material indexado, definindo o Processamento Natural de Linguagem como área da Computação dedicada a compreender e gerir a linguagem humana.

Este contraste é a evidência central de que o sistema cumpre o seu propósito: **responde quando tem suporte na fonte e admite quando não tem**, reduzindo as alucinações [5, 4].

```
Pergunta: qual é o atual presidente de portugal?

Resposta:
Não encontrei essa informação no conteúdo indexado.
=====
Pergunta: responda em uma frase curta o que é PNL

Resposta:
PNL significa Processamento Natural de Linguagem, uma área da Computação que visa entender e gerenciar linguagem humana.
=====
```

Figure 1: Funcionamento real do sistema: uma pergunta *não indexada* é recusada (em cima) e uma pergunta *indexada* é respondida com base no conteúdo (em baixo).

6. Utilização de IA Generativa no Desenvolvimento

Na preparação deste trabalho, o autor recorreu a modelos de linguagem de grande escala como auxílio na organização de conteúdos, na geração e revisão de código, na elaboração de rascunhos iniciais e na revisão linguística do texto. Todo o conteúdo produzido com recurso a estas ferramentas foi subsequentemente revisto e validado pelo autor, que assegurou o seu rigor técnico e coerência académica e assume plena responsabilidade pelo conteúdo final do presente trabalho.

7. Conclusão

Implementou-se um sistema RAG **completamente local**, capaz de responder a perguntas em linguagem natural sobre uma página Web indexada, sem qualquer chamada a APIs em operação e com

funcionamento *offline* após a ingestão. A combinação de *web scraping*, *Sentence-Transformers*, ChromaDB, LangChain e Llama 3.2 (via Ollama) reproduz fielmente a *stack* ensinada nas aulas de CLN da Universidade da Beira Interior, substituindo apenas os componentes baseados em API pelos seus equivalentes locais.

A decisão técnica mais relevante foi ancorar a geração no contexto recuperado e expor sempre as fontes, o que ataca diretamente o problema das alucinações [5]: o sistema responde com fundamento ou admite explicitamente a ausência de informação. A separação clara entre ingestão e operação permitiu, ainda, demonstrar de forma inequívoca o requisito de funcionamento sem Internet.

Como trabalho futuro, identificam-se três direções alinhadas com as aulas: combinar recuperação densa com pesquisa esparsa (BM25) numa abordagem híbrida; acrescentar *reranking* dos *chunks* antes da geração; e indexar múltiplas fontes (várias páginas ou PDFs), eventualmente caracterizando o corpus com modelação de tópicos (LDA).

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, *Attention Is All You Need*, Advances in Neural Information Processing Systems (NeurIPS), 2017. <https://arxiv.org/abs/1706.03762>
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, Proceedings of NAACL-HLT, 2019. <https://arxiv.org/abs/1810.04805>
- [3] A. Grattafiori et al. (Llama Team, AI @ Meta), *The Llama 3 Herd of Models*, arXiv preprint arXiv:2407.21783, 2024. <https://arxiv.org/abs/2407.21783>
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, *Retrieval-Augmented Generation for Large Language Models: A Survey*, arXiv preprint arXiv:2312.10997, 2023. <https://arxiv.org/abs/2312.10997>
- [5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, *Survey of Hallucination in Natural Language Generation*, ACM Computing Surveys, vol. 55, no. 12, 2023. <https://arxiv.org/abs/2202.03629>
- [6] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, *Dense Passage Retrieval for Open-Domain Question Answering*, Proceedings of EMNLP, 2020. <https://arxiv.org/abs/2004.04906>
- [7] N. Reimers and I. Gurevych, *Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation*, Proceedings of EMNLP, 2020. <https://arxiv.org/abs/2004.09813>
- [8] Ollama, *Ollama: Get up and running with large language models locally*, 2024. <https://ollama.com>
- [9] Chroma, *Chroma: the open-source AI application database*, 2024. <https://www.trychroma.com>
- [10] LangChain, *LangChain Documentation*, 2024. <https://python.langchain.com>
- [11] L. Richardson, *Beautiful Soup Documentation*, 2024. <https://www.crummy.com/software/BeautifulSoup/>